ASSIGNMENT 2.5

2303A51930

BATCH-30

Task 1: Refactoring Odd/Even Logic (List Version)

❖ Scenario:

You are improving legacy code.

❖ Task:

Write a program to calculate the sum of odd and even numbers in a list, then refactor it using AI.

CODE:

```
def sum_odd_even(numbers):
    odd_sum = 0
    even_sum = 0

    for num in numbers:
        if num % 2 == 0:
            even_sum += num
        else:
            odd_sum += num

    return odd_sum, even_sum

# Example usage
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
odd_sum, even_sum = sum_odd_even(numbers)
print(f"Odd sum: {odd_sum}")
print(f"Even sum: {even_sum}")
```

OUTPUT:

```
PS D:\AI Assisted coding>  & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\vinee\.vscode\extensions\ms-pyth
on.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '59461' '--' 'D:\AI Assisted coding\.vscode\2.5.py'
Odd sum: 25
Even sum: 30
PS D:\AI Assisted coding>
```

Task 2: Area Calculation Explanation

❖ Scenario:

You are onboarding a junior developer.

❖ Task:

Ask Gemini to explain a function that calculates the area of different shapes.

CODE:

```
import math

def calculate_area(shape, **dimensions):
    shape = shape.lower()

    if shape == "circle":
        # Formula: πr²
        return math.pi * (dimensions['radius'] ** 2)

    elif shape == "rectangle":
        # Formula: length * width
        return dimensions['length'] * dimensions['width']

    elif shape == "triangle":
        # Formula: 0.5 * base * height
        return 0.5 * dimensions['base'] * dimensions['height']

    else:
        return "Shape not supported."

# usage
print(calculate_area("circle", radius=5))
print(calculate_area("rectangle", length=10, width=4))
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' '
vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '58752' '--' 'D:\AI Assiste
.vscode\2.5.py'
78.53981633974483
40
PS D:\AI Assisted coding>
```

Task 3: Prompt Sensitivity Experiment

❖ Scenario:

You are testing how AI responds to different prompts.

❖ Task:

Use Cursor AI with different prompts for the same problem and observe code changes.

CODE:

```
# Problem: Calculate the sum of numbers in a list

# Prompt 1: "Write a function to sum numbers"
def sum_numbers_v1(numbers):
    total = 0
    for num in numbers:
        total += num
```

```
    return total

# Prompt 2: "Write a concise function to sum numbers"
def sum_numbers_v2(numbers):
    return sum(numbers)

# Prompt 3: "Write a function to sum numbers with error handling"
def sum_numbers_v3(numbers):
    try:
        if not isinstance(numbers, (list, tuple)):
            raise TypeError("Input must be a list or tuple")
        return sum(numbers)
    except TypeError as e:
        print(f"Error: {e}")
        return None

# Prompt 4: "Write a function to sum numbers using recursion"
def sum_numbers_v4(numbers):
    if len(numbers) == 0:
        return 0
    return numbers[0] + sum_numbers_v4(numbers[1:])

# Test all versions
if __name__ == "__main__":
    test_list = [1, 2, 3, 4, 5]
    print(f"v1: {sum_numbers_v1(test_list)}")
    print(f"v2: {sum_numbers_v2(test_list)}")
    print(f"v3: {sum_numbers_v3(test_list)}")
    print(f"v4: {sum_numbers_v4(test_list)}")
```

OUTPUT:

```
PS D:\AI Assisted coding>  d:; cd 'd:\AI Assisted coding'; & 'C:\Users\vinee\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users
vinee\.vscode\extensions\ms-python.debugpy-2025.19.2026012701-win32-x64\bundled\libs\debugpy\launcher' '56894' '--' 'D:\AI Assisted coding
.vscode\2.5.py'
v1: 15
v2: 15
v3: 15
v4: 15
PS D:\AI Assisted coding>
```

Task 4: Tool Comparison Reflection

❖ Scenario:

You must recommend an AI coding tool.

❖ Task:

Based on your work in this topic, compare Gemini, Copilot, and Cursor AI for usability and code quality.

SOLUTION:

Feature	Gemini Code Assist	GitHub Copilot	Cursor AI
Primary Strength	Large context (2M+ tokens) & GCP integration.	Reliability & seamless workflow.	Agentic multi-file refactoring.
Usability	Plugin-based; best if you use Google Cloud/Firebase.	Native to VS Code/JetBrains; "set it and forget it."	Standalone IDE (VS Code fork); requires migrating editors.
Code Quality	High quality for architectural logic & explanations.	Excellent for boilerplate & standard libraries (97% accuracy).	Superior for complex logic & cross-file consistency.
Unique Feature	Citations for suggested code; deep documentation search.	Workspace (Issue-to-PR) & Enterprise governance.	"Composer" mode for background agentic coding.